

Practical Light Clients for Committee-Based Blockchains

ABSTRACT

Light clients are gaining increasing attention in the literature since they obviate the need for users to set up dedicated blockchain full nodes. While the literature features a number of light client instantiations, most light client protocols optimize for long offline phases and implicitly assume that the block headers to be verified are signed by highly dynamic validators.

In this paper, we show that (i) most light clients are rarely offline for more than a week, and (ii) validators are unlikely to drastically change in most permissioned blockchains and in a number of permissionless blockchains, such as Cosmos and Polkadot. Motivated by these findings, we propose a novel practical system that optimizes for such realistic assumptions and achieves minimal communication and computational costs for light clients when compared to existing protocols. By means of a prototype implementation of our solution, we show that our protocol achieves a reduction by up to 90 and 40000 $\times$ (respectively) in end-to-end latency and up to 1000 and 10000 $\times$ (respectively) smaller proof size when compared to two state-of-the-art light client instantiations from the literature.

ACM Reference Format:

. 2024. Practical Light Clients for Committee-Based Blockchains. In *Proceedings of Annual Computer Security Applications Conference (ACSAC '24)*. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 INTRODUCTION

Light clients are programs that verify a digest of the blockchain's state with minimal storage, communication, and computation overhead. These clients are gaining increasing attention in the literature since they obviate the need for users to set up dedicated blockchain full nodes, store large amounts of data, and process all received information [13].

The literature features a number of proposals for light client implementations. For instance, the Bitcoin community provides the BitcoinJ [4], PicoCoin [21], and Electrum [16] clients, implementing the so-called Simple Payment Verification (SPV). In SPV, the light client has to verify the correctness of *each* block header it receives. This results in a linear communication complexity with respect to the number of block headers to be verified. Other more recent light client instantiations, such as [1, 23, 24], move away from Bitcoin's proof of work model and into a setting where the blockchain depends on signatures from elected committees. There, the light client proposals aim to optimize the (linear) communication complexity witnessed in the basic ("SPV-equivalent") mode by leveraging additional trust assumptions from the system (e.g., having a direct

communication link with an honest full node [1]) or by incurring additional computational overhead on the full nodes [23, 24]. As far as we are aware, Agrawal et al.'s PoPoS protocol [1] emerges as one of the most communication-efficient light client instantiations in the proof of stake setting requiring $O(\log m)$ messages to verify m consecutive block headers.

While these results greatly improve over the basic linear mode, we observe that most recent light client systems are designed with long offline phases in mind—trying to cater to the case where the number m of consecutive block headers to be verified is very large. In most popular cryptocurrencies, however, we observe that most light client requests are made by users who are rarely offline for more than a week. Moreover, we note that the most recent light client protocols implicitly assume that the block headers to be verified (by the light clients) are signed by continuously rotating keys. While this assumption is true for Ethereum and Algorand (where entirely new committees are frequently sampled from a large universe of nodes), this is not the case in most permissioned blockchains, such as Quorum [15] or Hyperledger Fabric [2], and in some permissionless committee-based blockchains such as Cosmos [17] and Polkadot [10].

Research Question and Solution. Motivated by these observations, we set forth in this paper to answer the following research question: *Can we design a light client protocol for committee-based blockchains that exhibits low communication and computation costs on both the light client and full nodes—without relying on additional trust assumptions?* To address this question, we focus on realistic blockchain deployments in which (1) a light client with short offline periods must be served most often and (2) the signers of block headers do not drastically change in the common case.

First, we ground the former assumption on the current usage patterns of light clients in popular cryptocurrencies, such as Bitcoin. Namely, our experiments on the Bitcoin blockchain reveal that more than half of all light clients connections to full nodes occur within at most 24 hours. Second, the latter assumption applies a priori to most permissioned blockchains as well as widely used permissionless blockchains. This is because permissioned blockchains like Hyperledger Fabric and Quorum are usually specifically designed for usage with small committees and rare validator changes for efficiency. On the other hand, our real-world evaluation of permissionless blockchains like Cosmos and Polkadot, shows that the membership changes that they exhibit are, in the common case, below a modest threshold relative to the total number ($< 10\%$).

By leveraging such (realistic) assumptions, we introduce a new light protocol that achieves minimal communication and computational overhead on the light clients and the full nodes in the standard security model (e.g., without requiring additional trust assumptions). We achieve this with the careful and novel utilization of *transitive signatures* [3], which have the useful property that long sequences of *consecutive* messages (in our case, information related to the evolution of the blockchain) can be checked with verifier work independent of the number of messages (here, blockchain epochs). To improve efficiency further, we exploit the

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ACSAC '24, December 9-13, 2024, Hawaii, U.S.A.

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

compatibility of transitive signatures with existing multi-signature aggregation techniques (formalized in Appendix A), which further allows compressing signatures issued by different blockchain validators. Overall, the light client’s security is based on the signature scheme’s proven security in conjunction with the majority-honesty assumption on validators, which is anyway already present in any committee-based system.

Contributions. In summary, we make the following contributions in this work:

Gaps in Existing Protocols: We analyze the practical usage patterns of light clients and show that over 50% of light client requests seen by full nodes concern offline phases of less than 2 hours. We also argue that validator sets are unlikely to change by large amounts in permissioned blockchains, and we empirically measured that if they changed at all in our observation period, then they only changed by up to $< 0.6\%$ at a time in Cosmos, 0% in XRP Ledger, and $< 7\%$ in Polkadot. Existing light client protocols have not been designed to take such operating conditions into account (Section 4.1).

Novel Construct: We introduce the first practical light protocol that achieves $O(1)$ communication complexity and verifier work in the common case of a large static validator subset and concretely low computational load on the full nodes without additional trust assumptions. Our design relies on the novel application of transitive signatures, which we generalize to multi-signatures for compatibility with the blockchain’s distributed setting (Section 5).

Prototype Implementation: We implement a prototype of the system (which we plan to release as open-source to aid further research in this area) and evaluate its performance compared to the PoPoS protocol [1] and a design proposed for usage in Polkadot from Ciobotaru et al. [14] (CSSV). We show empirically, by means of a prototype evaluation, that end-to-end latency for light client updates covering as many as 2^{16} epochs is over $90\times$ smaller than PoPoS and up to $41726\times$ smaller than CSSV. At the same time, the communication complexity (constant at under 200 bytes per static-quorum period) is up to $1000\times$ lower than PoPoS and up to $10000\times$ lower than CSSV (Section 6.2).

2 PRELIMINARIES

2.1 Blockchain Basics

A *blockchain data structure* is an append-only list of blocks. A block’s position in the blockchain is called its height. A block contains a header and a list of transactions. A transaction, abstractly, is an input command to a specified state machine. A (committee-based) *blockchain system* is a replicated state machine, maintained by a set of so-called *validators*, using a blockchain data structure and a (committee-based) consensus protocol. Consensus here specifically refers to Byzantine fault-tolerant atomic broadcast [12, 25]. At all times, an honest (super)majority assumption is placed on the validators.

Validator Reconfiguration, Epochs. The set of validator nodes may change over time. Such changes are also called reconfigurations. Blockchain systems define an epoch length as a number of

blocks such that reconfigurations are only allowed at block heights, which are multiples of the epoch length. In case of reconfiguration, a block with a special header field is output to announce the validator change. These end-of-epoch blocks are also signed by the (old) validators before the reconfiguration takes effect starting in the following epoch. Thus, the honest majority assumption on any particular set of validators is temporary—it only holds for a specific epoch.

Committee-Based Blockchains. We speak of a *committee-based* blockchain when the validators mutually know each other’s identities. This is in contrast to, say, Bitcoin’s protocol that supports an unknown set of nodes. However, this covers those *proof-of-stake* systems that select the participating nodes in a standard Byzantine consensus algorithm based on staked cryptocurrency. Such systems are widely used and include Ethereum and Polkadot (that is, their “finalized prefixes” [11]) and the Cosmos ecosystem based on Tendermint [9]. Committee-based blockchains also include *permissioned* blockchains like Hyperledger Fabric [2], where the assignment (and reconfigurations) of the set of validator nodes is decided by an external trusted party or procedure. In this paper, we treat such permissioned blockchains and proof-of-stake blockchains in parallel since both are committee-based.

Light Clients, Block Verification. All transactions in blocks from the initial block until a given block uniquely determine the blockchain system’s *state* at the given block. *Light clients* are programs that verify a digest of the blockchain’s state with minimal storage and computation overhead. Using the verified digest, they might then subsequently verify, say, parts of the state itself or verify that a particular transaction is included in a given block. It is standard that each block header includes the digest of the system state at that block. Thus, the light client’s task is verifying the block headers. A block header is considered valid if and only if there exist valid digital signatures over the block header from a sufficient number t of validators from the current validator set.

To receive updates about the blockchain, light clients do *not* directly communicate with validators. Rather, information about the blockchain’s state is forwarded to them by *full nodes*. Full nodes are highly available nodes that receive blocks from validators (or, via peer-to-peer messaging, from each other) and verify and store them. Unlike full nodes, light clients are generally not always online—they query full nodes on demand. Figure 1 illustrates the communication links in this setup.

For further context about light clients we refer to [13].

2.2 Cryptographic Building Blocks

We recall that standard digital signature schemes are defined by three algorithms KeyGen, Sign, Verify and are considered secure if they guarantee existential unforgeability under chosen message attacks (euf-cma). Aggregate signatures additionally have an algorithm Aggregate to combine signatures, and Verify is modified to be able to check a single signature for multiple messages and multiple sets of public keys.

BLS Signatures. Boneh, Lynn, and Shacham’s aggregate signature scheme (BLS) [6] is defined over three elliptic curve groups $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ of the same large prime order p , generators $g \in \mathbb{G}_1, \hat{g} \in \mathbb{G}_2$,



Figure 1: Overview of our setup (see Section 2.1).

a hash function $H : \{0, 1\}^* \rightarrow \mathbb{G}_2$ and a pairing function, that is, a bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$. KeyGen samples sk from $\mathbb{Z}_p$ and sets $pk = g^{sk}$. Sign on message x outputs $s = H(x)^{sk}$ and Aggregate($s_1, \dots, s_n$) computes $\prod_{i \in [n]} s_i$. Verify($(PK_1, \dots, PK_n), (x_1, \dots, x_n), s$) checks $e(g, s) = \prod_{i \in [n]} e(\prod_{pk \in PK_i} pk, H(x_i))$.

Transitive Signatures. Transitive signatures [19], intuitively, are used to sign edges of graphs and allow aggregating signatures from a single signer along paths. That is a signature on (x, y) and a signature on (y, z) can be combined into a valid signature for (x, z) , which can be verified without regard to y . The GapTS-2 scheme due to Bellare and Neven [3, Section 5.C] has the same KeyGen and Aggregate procedures as BLS. However, Sign computes signatures differently, namely as $s = (H(y)/H(x))^{sk}$, while Verify on input (x, z) checks $e(g, s) = e(pk, H(z)/H(x))$. Notably, verification complexity is independent of the path length.

2.3 Notations

In the sequel, we denote by $[n]$ the set $\{1, \dots, n\}$ and $[n, m]$ denotes $\{n, \dots, m\}$. In this paper, objects related to the i th block height or epoch are often indexed as, for example, $x^{(i)}$. Vectors are written as $\vec{x}$; $x[j]$ then denotes the j th entry of $\vec{x}$.

We further formalize the description of light clients given in Section 2.1. We denote by n the (maximum) validator committee size and write the set of validators in epoch i as $\mathcal{V}^{(i)}$. Regarding light client updates, we use the variable m for the number of epochs boundaries—that is, possible validator set changes—between the client’s starting block height and the most current height.

3 RELATED WORK

We now overview existing light client approaches for committee-based blockchains. Table 1 summarizes the core strengths and drawbacks. We remark that none of the existing protocols cater to the best-case scenario where the validator set is relatively static.

Sequential Verification (SV). We refer to the following standard baseline approach to light client verification as sequential verification (SV). At the start, the light client is initialized with the first committee’s public keys. To update to the current block header within the same epoch, the full node simply sends the block header and enough validators’ signatures to show that the block is valid. To update across epochs, the full node sends all end-of-epoch block headers with validator set changes. The light client then checks them in sequence, verifying the first one with the known keys and verifying each subsequent one with the public keys taken from the last one. Finally, the present epoch’s public keys are used to verify the most recent header.

PoPoS. The proofs of proof-of-stake (PoPoS) of [1] optimizes the SV approach as follows: instead of sequentially checking the validators’

signatures, the light client connects to multiple full nodes, at least one of which is assumed to be honest, and informs them of its local height h_1 . Then, each full node directly sends the most recent header. In the optimistic case, they all agree, and the update finishes immediately. Otherwise, if there is disagreement, the light client starts an interactive bisection protocol to identify the honest full node. Downsides to this approach are 1) that without an honest full node, the light client might accept a false state, 2) that even if there is an honest full node, a network-level adversary might prevent the light client from receiving the honest node’s responses and 3) the added latency of $O(\log(m))$ round-trip times due to the protocol’s interactivity.

Skipping Optimizations. Several proposals improve the SV approach via mechanisms that allow verifying multiple steps at once or “skipping ahead”. Examples are Tendermint [8], Algorand [20, Section 8] and Plumo [23]. Each skipping protocol introduces a parameter, which we uniformly denote as T , for the number of steps that can be skipped.

The Tendermint design [8] is based on the assumption that honest validators, even after removal from the committee, must continue to sign the next T end-of-epoch blocks and stay honest during this so-called trusting period. Thus, the Tendermint light client starting at height h_1 stores, in addition to the validators’ public keys, the height $h_1' < h_1$ when this set of validators was appointed¹. With this information, it is possible to quickly verify the epoch block just before $h_1' + T$ to obtain a new set of keys. Iterating, it is only necessary to check every T th block to reach the latest epoch, at which point the latest block is verified directly.

Algorand’s light client design [20, Section 8] follows a similar principle. There, every T th block contains a hash² of the list of public keys of a subset of the current validators. This subset is responsible for signing the next T th block, that is, the subset is implicitly trusted for T blocks in the future. This kind of trust assumption needed for Tendermint and Algorand might be backed by proof of stake and the threat of financial punishments for detectable misbehavior.

Cryptographic Proofs. Plumo [23] proposes the reliance on a cryptographic proof for T steps of sequential verification, implementing a proof-based version of “skipping ahead”. This approach is secure even if the trusting period is short, e.g., only one epoch. However, it comes with the drawbacks of 1) significant computational cost for proof creation and 2) added latency, as a new proof can only be created every T blocks.

¹We simplify for ease of explanation. In [8], the client stores the inauguration height for each validator individually rather than for the committee as a whole.

²Let it suffice to say that the specific use of (Merkle tree) hashes serves as a means to reduce the communication complexity *per update* from $O(n)$ to $O(\log(n))$.

	SV	PoPoS [1]	Tendermint [8]	Algorand [20]	Plumo [23], zkBridge (batched) [24]	CSSV [14]	Our solu- tion, Sec. 5
Best-case communication overhead (Ω)	–	1	–	–	–	–	1
Worst-case communication overhead (O)	mn	$\log(m)$	mn/T	$m \log(n)/T$	m/T	m	m
Secure if all full nodes are malicious	●	○	●	●	●	●	●
Epochs for which validators must <i>stay</i> majority-honest	1	1	T	T	1	1	1
Low computational load on full nodes / provers	●	●	●	●	○	●	●

Table 1: Comparison of light client protocols. SV stands for baseline sequential verification. The best case for PoPoS is that all queried full nodes agree. The best case for our scheme is that the set of validators did not change. T is a parameter for trust-based and proof-based skipping protocols. m is the number of epoch blocks the light client missed since last being online. n is the (maximum) size of a validator committee.

Finally, zkBridge [24] and CSSV [14] aim to reduce the per-step cost of sequential verification. Both proposals do not introduce additional trusted parties. In short, each step of the sequential verification is replaced with a cryptographic proof that the full node sends to the light client in place of the validators' $O(n)$ signatures. Specifically, zkBridge proposes using a general-purpose cryptographic proof, whereas CSSV presents an apparently more efficient scheme that is fine-tuned to systems where validators use BLS signatures over the elliptic curve BLS12-377. In the scenario where a light client returns after an offline phase, m proofs are needed. As m increases, this still has significant overall computational costs for proof generation and verification³.

4 PROBLEM STATEMENT

In this section, we motivate our research problem and outline our security model.

4.1 Motivation

We formulate our research problem based on the following observations of current real-world deployments.

Observation 1. Existing light client designs often *choose between* light client efficiency (1) after long offline phases ($m \rightarrow \infty$) or (2) for frequent small- m updates. Empirically, in the most popular cryptocurrency with widely used light clients, the majority of requests *that full nodes receive* correspond to small values of m equivalent to under 2 hours of missed blocks.

First, we note that the primary use cases of current light client designs differ. Some designs, such as Algorand and Plumo, efficiently allow clients to sync after long offline phases. As such, they particularly cater to initial bootstrapping, where a light client only has the initial blockchain state and must synchronize up to the present. However, they provide no benefit for “normal daily usage”, that is, for clients updating from starting heights less than T behind the present, since proofs are only created after every T th epoch. On the other hand, approaches such as zkBridge and CSSV purely focus on the case $m = 1$. They succeed in reducing the effect of n , which is desirable in the small- m regime, but suffer linear complexity

³[24] also supports batching, in which case only m/T proofs are needed (as in the “skipping” optimizations). As per the evaluation presented in [24], generating a proof that covers $T = 32$ blocks at once with $n = 128$ ECDSA signatures per block still takes 18 seconds on powerful hardware.

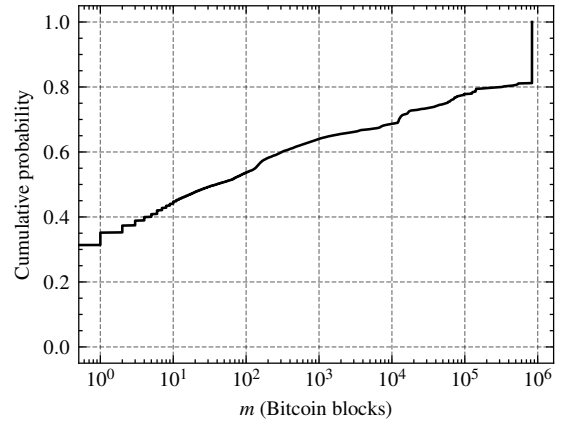


Figure 2: Empirical cumulative distribution function of measurements of m in Bitcoin.

when considering longer updates. (This refers to zkBridge without batching—with batching, the previous consideration applies since one would then have to wait until a batch is full before experiencing a benefit.)

The second part of Observation 1 stems from the following experiment which we conducted on Bitcoin to measure realistic absolute values of m that full nodes encounter. We opted to use Bitcoin since it is the most popular blockchain and was the first blockchain to support lightweight clients. To this end, we set up a Bitcoin full node for a period of 2 weeks and measured when the incoming light clients were last online, based on their so-called version and filterload messages which, respectively, reveal their last block height and identify them as light clients. Over the measurement period, 19458 samples were obtained. As shown in Figure 2, the results indicate that over a third of the light client requests queried for $m = 1$ block, that is, around 10 minutes in Bitcoin, and more than half asked for updates regarding $m = 12$, around 2 hours.

Observation 2. Existing light client designs achieve improvements over the mn communication overhead of the basic sequential verification (SV) either by exploiting additional trust assumptions or by incurring considerable computational load on the full nodes.

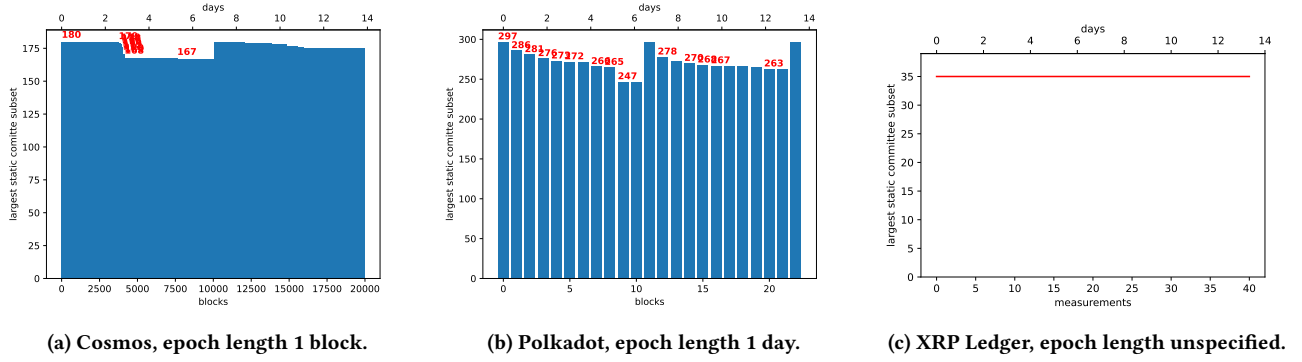


Figure 3: Maximum size of static validator subsets over a two-week period. Absolute values are indicated in red.

As laid out in Section 3, the surveyed designs all either (1) rely on honest full nodes for their security [1], (2) require validators to remain honest for longer periods [8, 20] or (3) introduce high computational costs for cryptographic proof generation [23, 24, 14]. When trying to use the designs in systems other than those for which they were originally developed, it might be difficult to determine whether the required trust assumptions are justified in that system or whether the proving costs are acceptable. For example, PoPoS [1] works under the assumption that a light client has a direct communication link to at least one honest full node. This assumption might be acceptable in some situations, but one might conceive of cases where it is not desirable. If, say, a mobile cryptocurrency wallet application was based on PoPoS, a network-level adversary might violate the assumption by dropping the honest node’s messages and might cause the light client to accept a false blockchain state—the wallet would display the wrong account balance or display false transactions⁴.

Regarding proof creation costs, we remark that the choice of the underlying cryptographic schemes plays an important role. For instance, Plumo [23] relies on validators using a specific signature scheme and hash function when they sign end-of-epoch blocks. zkBridge [24] relies on a more costly EdDSA signature scheme (even when using the new cryptographic proof system that optimizes for low overhead) when compared to CSSV [14]. Indeed, CSSV is fine-tuned to BLS signatures over a special elliptic curve and lets full nodes create proofs within hundreds of milliseconds—an improvement of two orders of magnitude—while the overall protocol structure is the same as zkBridge.

Observation 3. When considering existing designs that do not require honest full nodes, we note that the most costly aspect of the verification logic of such approaches is handling arbitrary validator set changes. However, we observe that in some widely used blockchains, large subsets of validators often remain across multiple epochs, and the relative size of validator changes is often less than 7%.

The first part of Observation 3 specifically regards the basic SV and the cryptography-based improvements to SV of Plumo,

⁴As mentioned in [1], such scenarios can be mitigated—though this might come with additional restrictions like added latency for a “dispute period” in which light clients will remain open to receiving contradicting claims.

zkBridge, and CSSV. We point out that in the case that validator changes are rare, a naive implementation of SV would lead to redundantly transmitting (or, at the very least, performing a verifier check over) n public keys for each epoch. A more efficient design would avoid this and, where possible, batch verify signatures with respect to the same public keys since that could be faster. Plumo, zkBridge, and CSSV are constructed to provide cryptographic proofs for the standard SV logic for arbitrary validator changes, but they do not seem to take such opportunities into account.

For the second part of this observation, we again point to an empirical analysis we conducted. Over the course of two weeks, we obtained the validator sets from public blockchain node endpoints for three widely used blockchain systems: Cosmos, Polkadot, and XRP Ledger. For each epoch, we then computed the size of the largest subset of the epoch’s validators, which was also in the following epoch’s validator set. This is shown in Figure 3. Notice how for Cosmos and Polkadot, subsets of 167 and 247 validators are present in *all* observed epoch’s validator sets. These are large majorities of the at most 180 and 300 total validators of Cosmos and Polkadot in the measured period. The largest relative change from one epoch to the next was under 7%. Further, as seen in Figure 3, in XRP Ledger, *all* validators remained in the validator set across the measurement period⁵. For permissioned systems like Hyperledger Fabric [2] and Quorum, where nodes are controlled by a few parties, we expect validator changes to be rare (similar to XRP Ledger) since reconfigurations (changes in the validator set) tend to be very costly in permissioned settings.

Research Question. We consider the aforementioned observations in conjunction. For instance, Observation 1 and Observation 3 suggest that full nodes are likely to frequently receive small- m light client update requests—where the update heights likely fall within static validator periods. This leads us to the following research question:

Is it possible to design a light client system for committee-based blockchains that (i) has a low concrete communication cost under realistic conditions, i.e., when m is small, and committees changes are

⁵In the particular case of XRP Ledger, nodes choose their own validator set. Our experiments on 5 randomly selected nodes show that nodes typically rely on a static set comprising, on average, 35 validators.

rare and (ii) that relies on minimal trust assumptions without (iii) incurring considerable computational overhead on full nodes?

4.2 Threat Model

Attacker. Throughout this paper, we assume a dynamic corruption of blockchain validators. That is, for each epoch i , each of the n validators $v \in \mathcal{V}^{(i)}$ is either in the set $\mathcal{V}_{\mathcal{A}}^{(i)}$ of corrupted validators or in the set of honest validators. The number of corrupted validators in any epoch i is bounded by f (e.g., $f \leq \frac{n-1}{3}$ in traditional Byzantine fault tolerant protocols [12])⁶:

$$\forall i : |\mathcal{V}_{\mathcal{A}}^{(i)} \cap \mathcal{V}^{(i)}| \leq f.$$

The used consensus protocol then determines, based on f , the minimum quorum size t , that is, the minimum number of validators which must sign a block. Corrupted parties can deviate arbitrarily from the protocols, except that corrupt validators cannot sign messages regarding previous epochs $< i$ in which they were honest. This models, e.g., the use forward-secure signatures where private keys have multiple components, one associated with each epoch, and honest validators delete the epoch-specific components at the end of the epoch.

Considering full nodes, we make no honesty assumptions and always consider them as possibly corrupted. Further, we do not make any explicit assumptions with respect to guaranteed message delivery between any parties. In particular, since an unreliable asynchronous (or adversarial) network environment might lead to the full nodes' messages to light clients getting dropped, unlike other works such as [1], we will not assume that there always exists an honestly responding full node to every light client request. Note that consequently, there can be no formal guarantee that any received state by the light client is the "most recent" one (new blocks might have come out before the full nodes' responses are received; full nodes themselves might not have received the most recent blocks yet). In practice, the chance that a full node returns stale data or does not respond at all might be lowered (heuristically) by connecting to multiple full nodes.

Security Goal. We say that there is one correct global state $s\mathcal{B}^{(h1)}$ of the underlying blockchain at each height $h1$ and one correct validator set $\mathcal{V}^{(i)}$ in each epoch i . Then, a light client at blockchain height $h1$ in epoch $e1$ stores in its local memory data $d^{(h1)}$ which, importantly, includes the current validators' ($\mathcal{V}^{(e1)}$) public keys (i.e., pk_v for $v \in \mathcal{V}^{(e1)}$) and a short digest of the blockchain's global state. In the LC.Update protocol, the client interacts with a set of full nodes $\mathcal{F}$, after which the client determines its new local state $d^{(h2)}$ corresponding to height $h2$ in epoch $e2$. Formally, let LCState be the function that given a global blockchain state $s\mathcal{B}^{(h1)}$ returns the (unique) corresponding correct light client state $d^{(h1)}$.

The following definition describes an update procedure as $(d^{(h2)}, \pi) \leftarrow \text{LC.Update}_{\mathcal{F}}(d^{(h1)})$, meaning a query is sent to full nodes in $\mathcal{F}$, each of sends (1) a response from which light client derives its new state $d^{(h2)}$ and (2) a proof π . The definition guarantees that a client starting from the correct state at height $h1$ either gets the light client state that corresponds to the global blockchain state

of the new height $h2$ or the verification of the received state fails (except with negligible probability).

Definition 1. We say that a light client scheme (LC.Update, LC.Verify) is secure if for all probabilistic polynomial-time adversaries $\mathcal{A}$, all choices of global blockchain states $s\mathcal{B}^{(h1)}$, $s\mathcal{B}^{(h2)}$, total blockchain lengths $h2$ and all heights $h1 \leq h2$, it holds that the success probability defined by

$$\Pr \left[\begin{array}{l} \text{LC.Verify}(d^{(h1)}, d, \pi) = 1 \\ \wedge \\ d \neq d^{(h2)} \end{array} \middle| \begin{array}{l} d^{(h2)} \leftarrow \text{LCState}(s\mathcal{B}^{(h2)}) \\ d^{(h1)} \leftarrow \text{LCState}(s\mathcal{B}^{(h1)}) \\ (d, \pi) \leftarrow \text{LC.Update}_{\mathcal{A}}(d^{(h1)}) \end{array} \right]$$

is negligible in the security parameter 1^λ , where $\text{LC.Update}_{\mathcal{A}}$ is the update protocol run with the adversary $\mathcal{A}(1^\lambda, h1, h2)$ choosing the behavior of all counterparties.

5 LIGHT CLIENT PROTOCOL

In this section, we introduce our light client protocol, which exhibits communication complexity asymptotically linear in the number of periods where a quorum of validators remains the same. In our design, validators sign epoch blocks using a signature scheme that is carefully tuned to later allow untrusted full nodes to *efficiently prove that a fixed validator subset remained static* to a light client without additional trust assumptions. Last but not least, we note that our design explicitly takes into account the observations of Section 4.1 and mostly optimizes for realistic deployment scenarios where the validator set is relatively static.

Before introducing our protocol, we start by describing a strawman solution and use it as a basis to introduce our full protocol later on.

5.1 Strawman Solution

Consider m completed consecutive epochs in which the client has been offline. For simplicity, we use epochs $1, \dots, m$ in this overview (such that the current epoch is $m+1$), and we focus on the interaction with one full node. The client's goal is to verify the latest signed digest of the blockchain's state—corresponding to a height that falls within the current epoch $m+1$ —, given that the client knows the keys belonging to the validator set $\mathcal{V}^{(1)}$ of epoch 1. Recall that in the standard design, at the end of every epoch i , validators publish signatures over the upcoming epoch's new validator set. For any epoch i , let $s_v^{(i)}$ denote this signature created by a validator v at the end of epoch i . We formalize the exact message being signed as $(i, \text{NextK}^{(i)})$. In the strawman design, $\text{NextK}^{(i)}$ is, naturally, exactly the list of all public keys of the next epoch's validators,

$$\text{NextK}^{(i)} = \{pk_v \mid v \in \mathcal{V}^{(i+1)}\}.$$

Static NextK. Let us assume first that there was no change at all in the validator set across all epochs in $[m]$. The main idea is that validators should treat the case of no change from the previous epoch differently when they output the end-of-epoch signatures. To serve a light client update, the full node will then aggregate all signatures $s_v^{(i)}$ into one signature S such that the client needs only to check S to verify that there was indeed no validator change in

⁶The consensus protocol determines, based on f , the minimum quorum size t , that is, the minimum number of validators which must sign a block.

the whole period. Now, what kind of special-case signature should be used in order to allow for efficient aggregation of this kind?

As a starting point, in order to ensure that S is really coupled with the correct epoch period, a natural approach is that $s_v^{(i)}$ is just a signature of “ i ”, that is $s_v^{(i)} = \text{Sign}(\text{sk}_v, i)$. With standard aggregatable BLS signatures, where aggregation simply means multiplication (see Section 2.2), the sketched protocol would result in

$$S = (H(1) \cdot \dots \cdot H(m))^{\text{ssk}}$$

where H is a hash function to an elliptic curve and ssk is the sum of the secret keys of the validators whose signatures were aggregated. While the client would now need to validate one signature only, the verification would still require computing a product of m different values. Thus, the light client’s effort grows linearly with m . This is already acceptable when m is small—a frequent occurrence, as seen in Observation 1—, but we can still do better and *circumvent* the “low m or high m ” tradeoff that existing designs make (Observation 1).

Reducing Verifier Work. To overcome this effort, we draw on so-called transitive signatures and construct signatures according to the scheme GapTS-2 of [3] (Section 2.2). That is, signatures are quotients such that $H(i)$ values cancel out when signatures are multiplied together. Only the start and end points remain. Concretely, using

$$s_v^{(i)} = (H(i)/H(i-1))^{\text{sk}_v}$$

creates the following upon aggregation:

$$\begin{aligned} S &= \prod_{i \in [m]} \prod_v s_v^{(i)} \\ &= \prod_{i \in [m]} (H(i)/H(i-1))^{\text{ssk}} \\ &= (H(m)/H(0))^{\text{ssk}}, \end{aligned}$$

for which verification is $O(1)$ as desired.

Dynamic NextK, Break Points. Next, we consider the case that the validator set changed. Formally, we have some index $j \in [m]$ such that $\text{NextK}^{(0)} = \dots = \text{NextK}^{(j-1)}$ but $\text{NextK}^{(j-1)} \neq \text{NextK}^{(j)}$. We call such an index a *break point*. We extend our approach above as follows: for any epoch i , any validator v produces the signature

$$s_v^{(i)} = \begin{cases} (H(i, \text{NextK}^{(i)})/H(i-1))^{\text{sk}_v} & \text{if } i \text{ is a break point} \\ (H(i)/H(i-1))^{\text{sk}_v} & \text{else.} \end{cases} \quad (1)$$

The role of $\text{NextK}^{(i)}$ here is twofold:

- (1) indicating that a change in the set of validators takes place, as well as
- (2) confirming the validity of the new validator set.

We now define $j_1, \dots, j_{\ell-1}$ as all break points in the range $1, \dots, m-1$ and furthermore $j_0 = 0$ and $j_{\ell} = m$. We divide the given period into subperiods $[j_0 + 1, j_1], \dots, [j_{\ell-1} + 1, j_{\ell}]$. For each subperiod $k \in [\ell]$, the full node generates the aggregated signature

$$S_k = \prod_{i \in [j_{k-1}+1, j_k]} \prod_v s_v^{(i)}.$$

As the intermediate quotients cancel out as before, it holds that

$$S_k = (H(j_k, \text{NextK}^{(j_k)})/H(j_{k-1}))^{\text{ssk}_k}$$

where ssk_k is the sum of secret keys of a subset of validators of this epoch range. For the last subperiod, we have

$$S_{\ell} = \begin{cases} (H(m, \text{NextK}^{(m)})/H(j_{\ell-1}))^{\text{ssk}_{\ell}} & \text{if } m \text{ is a break point} \\ (H(m)/H(j_{\ell-1}))^{\text{ssk}_{\ell}} & \text{else.} \end{cases}$$

Finally, the full node aggregates all signatures S_k and sends it to the client along with the break point indices and all the sets of public keys $\text{NextK}^{(j_1)}, \dots, \text{NextK}^{(j_{\ell})}$. Verification is clearly $O(\ell)$.

5.2 Final Protocol

Longest Running Quorum. The strawman design gives a protocol where the light client is $O(1)$ as long as the full validator set remains exactly the same. We now move to the setting where only a static quorum is required. This allows us to cater to minor changes in the validator set, like in Cosmos and Polkadot (see Figure 3). More specifically, we assume that validators keep track of the churn in the system (which is typical in most consensus deployments) and identify a subset of nodes that are most likely to remain static over time. Suppose that each validator in epoch i keeps a record of the number of epochs for which every (other) validator has already been in the set. Each validator can then determine the unique subset $Q^{(i)} \subset \mathcal{V}^{(i)}$ that has constituted a quorum for the most consecutive epochs so far (compared to all other subsets). Ties can be broken using any deterministic rule to ensure that this *longest running quorum* is unique⁷.

From $\mathcal{V}$ to $\mathcal{Q}$. With this, the only change the full protocol makes to the strawman is that we re-define $\text{NextK}^{(i)}$ as a single value, the next longest running quorum’s combined public key:

$$\text{NextK}^{(i)} = \prod_{v \in Q^{(i+1)}} \text{pk}_v.$$

Consequently, break points are also based on $Q^{(i)}$ rather than $\mathcal{V}^{(i)}$. This guarantees that valid signatures of the type $H(i)/H(i-1)$ issued by all validators in $Q^{(i)}$ exist as long as the set $Q^{(i)}$ remains a quorum. Furthermore, such signatures *only* exist as long as $Q^{(i)}$ remains a quorum due to the honest majority assumption, which ensures security. All of these signatures can be aggregated and verified in $O(1)$, making the protocol as efficient as desired.

With no further modifications, we end up with the simple 2-move protocol in Figure 4, which we briefly walk through. First, the light client sends the starting epoch $e1$ (①). Next, the full node must find the break points $j_1, \dots, j_{\ell-1}$ between $e1$ and the current epoch $e2$, as well as the subsets $Q^{(j_1+1)}, \dots, Q^{(j_{\ell-1}+1)}$ and aggregate the signatures of all validators in these sets (②, ③). The full node then sends over (④) the break points, the combined public keys of all longest-running quora and the most recent block header. To conclude, the light client verifies that each longest-running quorum signed the next one’s keys (⑤) and uses the final quorum’s keys

⁷In full generality, the protocol works for any unambiguous definition of the longest running quorum.

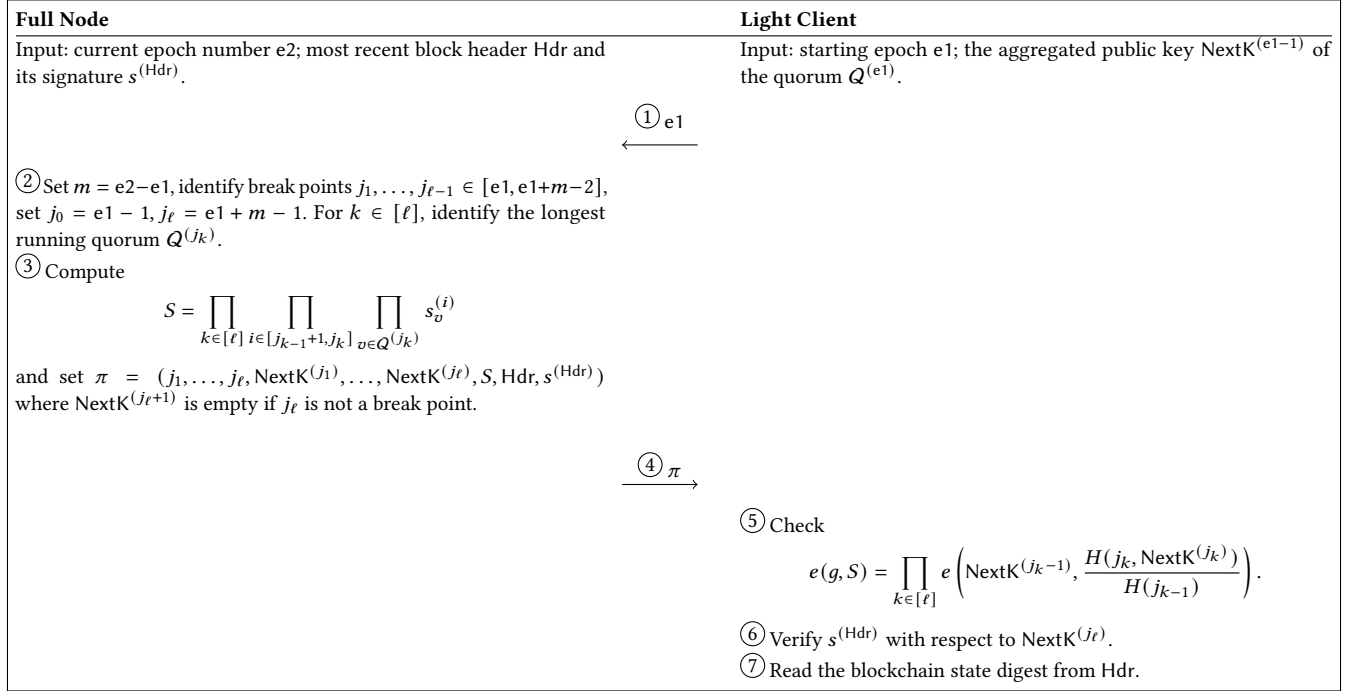


Figure 4: Light client protocol for an update from epoch $e1$ to the current epoch $e2$.

to verify the most recent header, from which the newest state is obtained (⑥, ⑦).

Efficiency. Clearly, the communication complexity and verifier work in our protocol are linear in the number ℓ of static-quorum periods and independent of due to restricting the light client's checks to the aggregated public key of a valid quorum. The full node only performs m elliptic curve multiplications to aggregate signatures.

As mentioned in Observation 2, we note that existing light client protocols either put a high computational load on full nodes or require a relaxed trust model to be secure. Ours, on the other hand, is secure in a strong trust model (analyzed next) and simultaneously obviates the need for full nodes to compute expensive full-fledged zero-knowledge proofs in favor of mere multiplications.

5.3 Security Analysis

To analyze our construct's security, we first show that a certain building block is implicit in our protocol description which we refer to as a transitive multi-signature scheme and for which we can prove a meaningful security notion independently. Then, the overall security will follow from the transitive multi-signature's security as well as the system's inherent trust assumptions (that is, without introducing new ones).

While transitive signatures are known—recall from Section 2.2 that they allow for $O(1)$ verification of sequences of messages that are consecutive in a well-defined sense, only based on the start and end points, we are not aware of transitive signatures with multiple signers' public keys having been considered before. We formally

define this generalized functionality in Appendix A as a transitive multi-signature.

Proposition 1. The signature scheme implicitly defined by Equation (1), Step ③ and Step ⑤ is a secure transitive multi-signature scheme.

PROOF. (Sketch.) Observe that the signing, aggregation, and verification functions used in the protocol are exactly the same as those of the scheme in Figure 6, proven secure in Appendix A (simply substituting the variable x with $i - 1$, y with i or $(i, NextK^{(i)})$, etc.). Due to lack of space, we defer the full proof to Proposition 3 in Appendix A. $\square$

By leveraging Propositions 1 and 3 and the per-epoch majority honesty of validators—an assumption already present in most blockchain systems—we can now prove the security of our light client.

Proposition 2. The protocol of Figure 4 is a secure light client update protocol as per Definition 1.

PROOF. (Sketch.) To satisfy Definition 1, we argue that if the update protocol succeeds, then the light client is at the correct blockchain state in the epoch specified in block header Hdr (in epoch $e2$). Specifically, if the check in step ⑤ succeeds, then the light client can be sure that the validators in $Q^{(e1-1)}$ have indeed signed $NextK^{(j_1)}$ etc. This is due to this check exactly implementing the transitive signature verification, which fulfills exactly this security notion (Propositions 1 and 3). By the per-epoch majority-honesty assumption on validators, it follows that the light client

obtains the *true* values for $\text{NextK}^{(jk)}$. In particular, the light client gets the aggregated public key of the epoch e_2 quorum $Q^{(e_2)}$. Finally, due to the security of standard signature verification in step ⑦ and the honesty of $Q^{(e_2)}$ regarding all blocks in epoch e_2 , the light client is sure to obtain a correct header. Thus, the blockchain state digest with which the light client updates its local storage is also a correct one. Due to lack of space, we include the full proof in Appendix B. $\square$

6 IMPLEMENTATION & EVALUATION

In this section, we detail the prototype implementation of our solution, and we evaluate its performance compared to PoPoS [1] and CSSV [14].

6.1 Implementation Setup

We implemented a prototype light client system in Go. In our implementation, we relied on gnark-crypto's [7] implementation of the elliptic curve BLS12-377⁸. On this curve, a single signature takes up 96 bytes, and a public key takes up 48 bytes. Since our main goal was to evaluate the performance of our approach compared to competing approaches, in our prototype implementation, the full node has access to a generated chain of validator public keys and signatures that it feeds to the light client. Specifically, we generated 2^{16} sets of $n = 128$ public keys, with each set standing for one epoch. To model the setting that validator changes are always possible but often have a static quorum, the epochs are divided into subperiods of a fixed number of epochs where such a static quorum is present for the whole subperiod. In the experiments below, this length was set as 2000⁹. The full node also had signatures, valid under these t static public keys, over each set of the next n keys. Additionally, we performed measurements where the full node's work is helped by realistic pre-computations: namely, where the signatures from each epoch have already been pre-aggregated (i.e., for each epoch i , $\prod_v s_v^{(i)}$ was already available in step ③ of Figure 4). This strategy is marked with "**precomp.**" in plots in Figure 5 below.

We deployed our full node implementation on a dual 64-core AMD EPYC 7742 machine, clocked at 2.25 GHz, with 1024 GB of RAM. The light client ran on an 8-core Apple M2 laptop. We conducted our experiments within a wide area network (WAN) with a mean delay of 30 ms and a mean bandwidth of 16 megabits per second. We argue that such a WAN setting faithfully emulates the deployment setting of many light client users around the globe. In our setup, each client invokes an operation in a closed loop; a client may have at most one pending operation. For different values of $m \leq 2^{16}$, we measure the end-to-end latency observed by the client for a successful update from height 1 to height m , as well as the proof generation time measured at the full node. Each data point in our plots is averaged over 50 independent trials.

Baselines. For a baseline comparison, we considered the light client protocols of [1] ("**PoPoS**") and of [14] ("**CSSV**"). We choose PoPoS due to it being the only asymptotically sublinear protocol and CSSV because it seems to be the most efficient one in the same

trust model as ours. For PoPoS, we used the benchmarking code provided alongside the paper [1] with the following modifications:

- (1) While PoPoS is a multi-prover protocol, we run 8 different full nodes on independent CPU cores on the *same* server machine so that the light client has a uniform network connection to all full nodes;
- (2) We set the committee size parameter $n = 128$ as above and set the internal "tree degree" parameter to 100 which is recommended in [1];
- (3) We add code to measure the total time spent by the full node on computing its responses; below, we report the average of the 8 full nodes.

Regarding CSSV's (single-server, like ours) protocol, in lieu of an implemented networked light client benchmark, we used the local evaluation tool part of the open-source implementation of [14] (unmodified). The tool runs on a single machine to create a chain of validator sets and corresponding signatures and proofs. The time to create and verify light client proofs is output for each individual validator set. Below, as "end-to-end latency" for an update from epoch 1 until epoch i , we use the sum of all the proving times plus the verification times of epochs from 1 to i .

Note that, in our experiments, the meaning of absolute m values depends on the actual instantiation: in terms of wall-clock time, its meaning depends on the definition of an epoch in a given system (measured in a number of blocks) as well as the time for blocks to be created. For example, 2^{16} Cosmos epochs mean, with around 6 seconds per block and 1 block per epoch, around 4 days and 12 hours. On the other hand, in Polkadot epochs, 2^{16} would instead amount to around 179 years because an epoch spans one day.

6.2 Performance Evaluation

Here, we evaluate the proof size, the latency, and the full node costs incurred by our solution when compared to PoPoS [1] and CSSV [14]. The full results are depicted in Figure 5.

Proof Size. The comparison of proof sizes is shown in Figure 5a. In the parameter regime of this evaluation, our scheme sends a proof with a size of 152 bytes per static-quorum period. This proof is composed of (1) the new epoch number (8 bytes), (2) an aggregated public key (48 bytes) and (3) the aggregated signature (96 bytes). Since short updates ($m < 2000$) in our implementation setup fall into such periods, this is exactly the number of communicated bytes in these cases. With larger m , proof size increases linearly up to around 200 kilobytes for 2^{16} epochs. On the other hand, PoPoS' interactive multi-round multi-server protocol required more data to be transmitted overall: almost uniformly the same amount of data for all m , namely around 200 kilobytes. Thus, our solution has a smaller proof size by more than 3 orders of magnitude when considering a single static-validator period—a gap which narrows to 2 orders of magnitude as m increases. For CSSV, since static-committee periods are not optimized, every epoch has a (redundant) proof of size 976 bytes. In direct comparison, our solution has a smaller size by at least $5.8\times$ and in the experiments with $m = 2^{16}$ by as much as 4 orders of magnitude.

⁸The curve BLS12-377 is chosen to enable a fair comparison for comparison with CSSV, whose prototype implementation relies on this curve. Our scheme and PoPoS are agnostic to the curve choice.

⁹As seen in Figure 3, Cosmos often has longer periods with a static quorum.

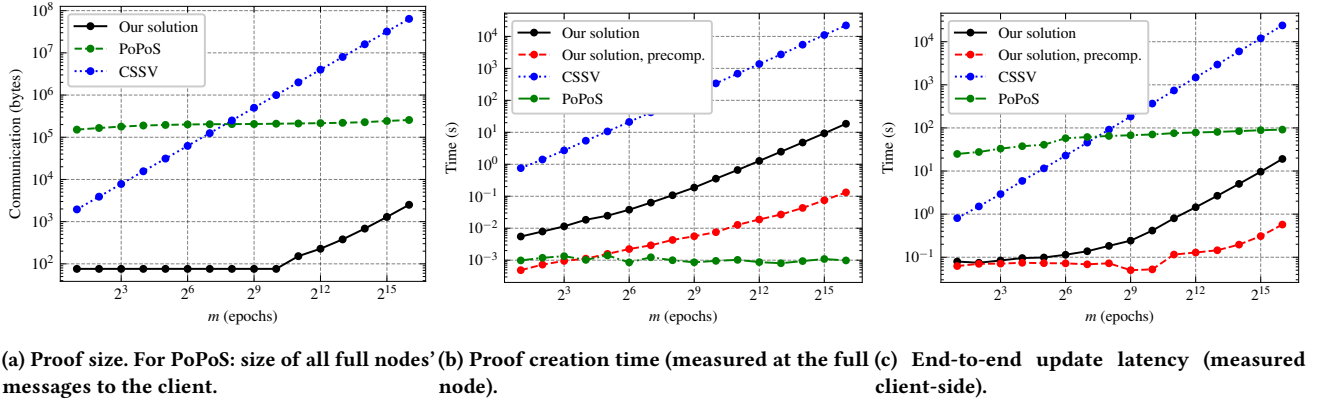


Figure 5: Evaluation results of our solution when compared to PoPoS [1] and CSSV [14] in our implementation setup.

Full Node Computations. The comparison of proof creation timings is shown in Figure 5b. With our solution, the full node's computations consist of $mn = m \cdot 128$ elliptic curve multiplications. In our setup, this took under 100 milliseconds for $m < 2^8$ and around 20 seconds for $m = 2^{16}$. The pre-computation strategy needs only m multiplications since the first aggregation step over n has been done previously. Figure 5b confirms that the pre-computations reduce the full node's computation time to under 100 milliseconds when $m \leq 2^{15}$. By comparison, PoPoS full nodes have lower requirements since that protocol's core operations are logarithmic in m (albeit at the cost of additional trust assumptions; see Section 3). We contrast this with CSSV, where computing a proof took around 200 milliseconds for one epoch. For instance, we note that, in the time taken to compute a CSSV proof for one epoch, our solution with pre-computations allows creating a proof that covers over 2^{16} epochs, a 65536 $\times$ improvement or 4 orders of magnitude. When aggregating from scratch, the improvement is still 2048 $\times$.

Latency. The comparison of end-to-end client latencies is shown in Figure 5c. First, however, note that these values are lower bounded by prover computation time plus network latency (except for CSSV where the numbers have no networking component). We can thus conclude from the prover time shown in Figure 5b alone that our solution's latency is likely dominated by the implementation setup's round trip time of 30 milliseconds for m as large as 32 epochs or, with pre-computations, for m as large as 2^{13} . Indeed, as seen in Figure 5c, for smaller m , our solution's end-to-end latency is close to constant. After that, we find linear increases in accordance with the increasing prover time. Overall, our scheme's latency is below one second for m up to 2^{11} or, with pre-computations, up to 2^{16} . We contrast this with PoPoS where, likely due to the multiple rounds of network communications, even an update for $m = 2$ epochs takes over 20 seconds. Asymptotically, however, PoPoS' latency scales attractively as the case $m = 2^{16}$ takes only around 100 seconds. In this case of $m = 2^{16}$, our solution with pre-computations still gives a latency improvement of around 90 $\times$ compared to PoPoS. Finally, we observe that our solution's end-to-end latency, both with and without pre-computations, already at $m = 2$, is around 10 $\times$ lower than CSSV's. The gap increases up to an improvement

of, respectively, 3 orders of magnitude without pre-computations and 41726 $\times$ with pre-computations.

7 CONCLUSION

In this paper, we presented a novel light client protocol for committee-based blockchains that exhibits low communication and computation costs on both the lightweight client and full nodes. Our protocol is mostly geared to practical deployment settings where (1) lightweight clients are not offline for long periods of time or (2) the signers of block headers have a long-term static core quorum. Our protocol leverages transitive signatures [19] to verify long sequences of consecutive block headers—independently of the number of blockchain epochs. To improve efficiency further, we argue that the compatibility of transitive signatures with existing multi-signature aggregation techniques allows for better compression of signatures issued by the blockchain validators.

We validated the performance of our protocol by means of prototype implementation in a realistic WAN setting. Our evaluation results show that our protocol achieves a reduction by up to 41726 $\times$ in end-to-end update latency and around 1000 $\times$ smaller proof size when compared to the PoPoS [1] and CSSV [14] designs.

REFERENCES

- [1] Shreshth Agrawal et al. "Proofs of Proof-Of-Stake with Sublinear Complexity". In: *5th Conference on Advances in Financial Technologies, AFT 2023, October 23-25, 2023, Princeton, NJ, USA*. Ed. by Joseph Bonneau and S. Matthew Weinberg. Vol. 282. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023, 14:1–14:24. doi: 10.4230/LIPICS.AFT.2023.14.
- [2] Elli Androulaki et al. "Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains". In: *Proceedings of the Thirteenth EuroSys Conference*. EuroSys '18. Porto, Portugal: Association for Computing Machinery, 2018. ISBN: 9781450355841. doi: 10.1145/3190508.3190538.
- [3] Mihir Bellare and Gregory Neven. "Transitive signatures: new schemes and proofs". In: *IEEE Trans. Inf. Theory* 51.6 (2005), pp. 2133–2151. doi: 10.1109/TIT.2005.847697.
- [4] BitcoinJ. 2018. URL: <https://bitcoinj.github.io>.
- [5] Alexandra Boldyreva. "Threshold Signatures, Multisignatures and Blind Signatures Based on the Gap-Diffie-Hellman-Group Signature Scheme". In: *Public Key Cryptography — PKC 2003*. Ed. by Yvo G. Desmedt. Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, pp. 31–46. ISBN: 978-3-540-36288-3.
- [6] Dan Boneh, Ben Lynn, and Hovav Shacham. "Short Signatures from the Weil Pairing". In: *J. Cryptol.* 17.4 (2004), pp. 297–319. doi: 10.1007/S00145-004-0314-9.
- [7] Gautam Botrel et al. *ConsenSys/gnark-crypto: v0.11.2*. Version v0.11.2. Jan. 2023. doi: 10.5281/zenodo.5815453.

- [8] Sean Braithwaite et al. *A Tendermint Light Client*. arXiv. 2020. DOI: 10.48550/ARXIV.2010.07031.
- [9] Ethan Buchman, Jae Kwon, and Zarko Milosevic. *The latest gossip on BFT consensus*. arXiv. 2018. DOI: 10.48550/ARXIV.1807.04938.
- [10] Jeff Burdges et al. *Overview of Polkadot and its Design Considerations*. arXiv. <https://doi.org/10.48550/arXiv.2005.13456>. 2020. arXiv: 2005.13456 [cs.CR]. DOI: 10.48550/arXiv.2005.13456.
- [11] Vitalik Buterin et al. *Combining GHOST and Casper*. arXiv. 2020. DOI: 10.48550/arXiv.2003.03052.
- [12] Miguel Castro and Barbara Liskov. "Practical Byzantine Fault Tolerance and Proactive Recovery". In: *ACM Trans. Comput. Syst.* 20.4 (Nov. 2002), pp. 398–461. ISSN: 0734-2071. DOI: 10.1145/571637.571640.
- [13] Panagiotis Chatzigiannis, Foteini Baldimtsi, and Konstantinos Chalkias. "SoK: Blockchain Light Clients". In: *Financial Cryptography and Data Security: 26th International Conference, FC 2022, Grenada, May 2–6, 2022, Revised Selected Papers*. Grenada, Grenada: Springer-Verlag, 2022, pp. 615–641. ISBN: 978-3-031-18282-2. DOI: 10.1007/978-3-031-18283-9_31.
- [14] Oana Ciobotaru et al. *Accountable Light Client Systems for PoS Blockchains*. Cryptology ePrint Archive, Paper 2022/1205. Code available at <https://github.com/w3f/apk-proofs>. 2022. URL: <https://eprint.iacr.org/2022/1205>.
- [15] ConsensusSys. *ConsensusSys/Quorum: v24.4.0*. Version v24.4.0. 2024. URL: <https://github.com/ConsensusSys/quorum>.
- [16] *Electrum*. 2018. URL: <https://electrum.org/#home>.
- [17] Jae Kwon and Ethan Buchman. *Cosmos. A Network of Distributed Ledgers*. 2016. URL: <https://github.com/cosmos/cosmos/blob/master/WHITEPAPER.md>.
- [18] Yehuda Lindell. "Parallel Coin-Tossing and Constant-Round Secure Two-Party Computation". In: *Journal of Cryptology* 16.3 (2003), pp. 143–184. DOI: 10.1007/s00145-002-0143-7.
- [19] Silvio Micali and Ronald L. Rivest. "Transitive Signature Schemes". In: *Topics in Cryptology - CT-RSA 2002, The Cryptographer's Track at the RSA Conference, 2002, San Jose, CA, USA, February 18-22, 2002, Proceedings*. Ed. by Bart Preneel. Vol. 2271. Lecture Notes in Computer Science. Springer, 2002, pp. 236–243. DOI: 10.1007/3-540-45760-7_16.
- [20] Silvio Micali et al. "Compact Certificates of Collective Knowledge". In: *2021 IEEE Symposium on Security and Privacy (SP)*. 2021, pp. 626–641. DOI: 10.1109/SP40001.2021.00096.
- [21] *PicoCoin*. 2018. URL: <https://github.com/jgarzik/%20picocoin>.
- [22] Thomas Ristenpart and Scott Yilek. "The Power of Proofs-of-Possession: Securing Multiparty Signatures against Rogue-Key Attacks". In: *Advances in Cryptology - EUROCRYPT 2007, 26th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Barcelona, Spain, May 20-24, 2007, Proceedings*. Ed. by Moni Naor. Vol. 4515. Lecture Notes in Computer Science. Springer, 2007, pp. 228–245. DOI: 10.1007/978-3-540-72540-4_13.
- [23] Psi Vesely et al. "Plumo: An Ultralight Blockchain Client". In: *Financial Cryptography and Data Security: 26th International Conference, FC 2022, Grenada, May 2–6, 2022, Revised Selected Papers*. Grenada, Grenada: Springer-Verlag, 2022, pp. 597–614. ISBN: 978-3-031-18282-2. DOI: 10.1007/978-3-031-18283-9_30.
- [24] Tiancheng Xie et al. "ZkBridge: Trustless Cross-Chain Bridges Made Practical". In: *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS '22*. Los Angeles, CA, USA: Association for Computing Machinery, 2022, pp. 3003–3017. ISBN: 9781450394505. DOI: 10.1145/3548606.3560652.
- [25] Maofan Yin et al. "HotStuff: BFT Consensus with Linearity and Responsiveness". In: *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing, PODC '19*. Toronto ON, Canada: Association for Computing Machinery, 2019, pp. 347–356. ISBN: 9781450362177. DOI: 10.1145/3293611.3331591.

A TRANSITIVE MULTI-SIGNATURES

In this section, we put forth a notion of transitive multi-signatures. We show that constructing a transitive multisignature from Bellare and Neven's GapTS-2 [3, Section 5.C] scheme works analogously to the construction of multi-signatures from BLS signatures.

A.1 Definitions

A standard transitive signature consists of three algorithms KeyGen, Sign and Verify and makes it possible to combine the same signer's signatures over messages of the form (x, y) and (y, z) into a valid signature over (x, z) . Such messages are best interpreted as edges of an undirected graph G , meaning that aggregation is possible along *paths*. The formal security notion for transitive signatures is dubbed transitive unforgeability under adaptive chosen message

attack (tuf-cma) and demands that no probabilistic polynomial-time adversary with access to a signing oracle for the public key pk can, with non-negligible probability, output x, y and a signature s such that (1) $\text{Verify}(pk, x, y, s)$ accepts even though (2) there is no path from x to y in G . Here, queries to the signing oracle add nodes and edges to G .

Transitive Multi-Signatures. The following definition is inspired by existing notions of signature aggregation [cf. 5], which allows signatures from different sets of signers on different messages to be combined into a single one. In practical systems, it is typical to demand a key registration phase in which so-called proofs of possession are checked in order to prevent certain attacks [cf. 22]. For simplicity, our analysis will model this usage using a so-called knowledge of secret key assumption (see below), and we hence omit explicitly specifying proof of possession algorithms as part of the signature scheme definition¹⁰.

Definition 2 (Transitive Multi-Signature). A transitive multi-signature consists of the following algorithms.

- $\text{KeyGen}(1^\lambda) \rightarrow (sk, pk)$. For a security level λ , outputs a secret key and a public key.
- $\text{Sign}(sk, x, y) \rightarrow s$. For $x, y \in \{0, 1\}^*$, outputs a signature s .
- $\text{Aggregate}(s_1, \dots, s_n) \rightarrow S$. For signatures $s_1, \dots, s_n$, outputs an aggregated signature S .
- $\text{Verify}((PK_1, \dots, PK_n), ((x_1, z_1), \dots, (x_n, z_n)), S) \rightarrow 0/1$. For lists of public keys $PK_1, \dots, PK_n$, for $x_1, z_1, \dots, x_n, z_n \in \{0, 1\}^*$ and for a signature S , accepts or rejects.

A transitive multi-signature supports *secure aggregation* if no probabilistic polynomial-time adversary with access to a signing oracle for the public key pk can, with non-negligible probability, output sets of public keys $PK_1, \dots, PK_n$, a signature s and $(x_1, z_1), \dots, (x_n, z_n)$ such that

- (1) $\text{Verify}((PK_1, \dots, PK_n), ((x_1, z_1), \dots, (x_n, z_n)), s)$ accepts,
- (2) $\exists j \in [n]$ such that $pk \in PK_j$ and
- (3) there is no path between x_j and z_j in the graph created from signing queries to the pk oracle.

A.2 Construction

The scheme based on GapTS-2 is shown in Figure 6. GapTS-2 itself consists of similar key generation and signing algorithms as in Figure 6 and only restricts $n = 1$ and $|PK_1| = 1$ in Verify. It is shown in [3] that GapTS-2 is a secure standard transitive signature scheme under the one-more computational Diffie-Hellman assumption in the random oracle model. Based on this, we now prove that the multi-signature is secure as well.

Proposition 3. The scheme of Figure 6 is a secure transitive multi-signature.

PROOF. We follow similar steps as in the security proof for BLS multi-signatures from [5, Section 4.3]. That is, we assume the existence of an adversary A against aggregation security and will derive a tuf-cma adversary B (Appendix A.1) against GapTS-2.

For simplicity and ease of presentation, we make a *knowledge of secret key* assumption whereby the attacker must directly output the secret keys belonging to the (other) public keys in its output

¹⁰For the scheme presented below, the same techniques as for BLS signatures apply.

KeyGen(1^λ)	Sign(sk, x, y)	Aggregate($s_1, \dots, s_n$)
$sk \leftarrow \mathbb{Z}_p$ $pk = g^{sk}$ return (sk, pk)	$s = \left(\frac{H(y)}{H(x)} \right)^{sk}$ return s	$S = \prod_{i \in [n]} s_i$ return S
Verify ((PK ₁ , ..., PK _n), ((x ₁ , z ₁), ..., (x _n , z _n)), S)		
for $i \in [n]$, $apk_i = \prod_{pk \in PK_i} pk$		
Check $e(g, S) = \prod_{i \in [n]} e\left(apk_i, \frac{H(z_i)}{H(x_i)}\right)$		

Figure 6: Transitive multi-signature scheme.

besides pk. This models a (real-world) key registration model, where verification would additionally require each public key to have a separate *proof of knowledge* of the corresponding secret key [cf. 22]. Then, since an attacker A would also have to prove knowledge of all the secret keys it used, by a standard definition of a proof of knowledge [18], there must exist a polynomial-time extractor algorithm that would interact with A to obtain the secrets. The explicit knowledge assumption abstracts this step.

Now, suppose B received the public key pk in the tuf-cma experiment. B would send all parameters to A , forward all of A 's queries to the tuf-cma challenger, and forward the responses to A . Eventually, A would output $PK_1, \dots, PK_n, x_1, z_1, \dots, x_n, z_n$ and s . Say j is such that $pk \in PK_j$. Let $SK_1, \dots, SK_n$ be the lists of secret keys corresponding to the public keys, which B could obtain as per the knowledge of secret key assumption—except for the secret key sk^* corresponding to pk. If s passed verification, then it would hold that

$$s = \prod_{i \in [n]} \left(\frac{H(z_i)}{H(x_i)} \right)^{\sum_{sk \in SK_i} sk}.$$

Hence, B divides s by

$$\left(\prod_{i \in [n] \setminus \{j\}} \left(\frac{H(z_i)}{H(x_i)} \right)^{\sum_{sk \in SK_i} sk} \right) \cdot \left(\frac{H(z_j)}{H(x_j)} \right)^{\sum_{sk \in SK_j \setminus \{sk^*\}} sk}$$

and obtains $(H(z_j)/H(x_j))^{sk^*}$. This constitutes a valid forgery for GapTS-2, which contradicts tuf-cma security and completes our proof. $\square$

B LIGHT CLIENT SECURITY

Proposition 2. The protocol of Figure 4 is a secure light client update protocol as per Definition 1.

PROOF. To formally map the security definition of Section 4.2 to the protocol description, let us first make explicit that the light client's local storage at a height h_1 consists of the blockchain state digest (that is, the digest of $s\mathcal{B}^{(h_1)}$), the blockchain epoch number e_1 and the aggregated public key $\text{NextK}^{(e_1-1)}$ of the validator quorum $\mathcal{Q}^{(e_1)}$.

Next, we convince ourselves that the validators, as per Equation (1), are implicitly using exactly the transitive signature scheme for which we proved secure aggregation in Appendix A. Specifically, the inputs to Sign in epoch i are $(i-1, i)$ or, if i is a break point, $(i-1, (i, \text{NextK}^{(i)}))$. Likewise, the full nodes operations in Step ③ are equivalent to the Aggregate algorithm and the light client performs, in Step ⑤, a version of the Verify algorithm where the quantity denoted as apk_i in Appendix A is already given as a precomputed value, namely $\text{NextK}^{(e_1-1)}$. By the validators' honest majority assumption (since they are the ones who originally supply this precomputed value), it is secure to use this value.

Thus, we can rely on the security properties of the signature shown in Proposition 3. In particular, we are now ready to prove what our security definition requires: that when verification passes, then the final state is a correct state except with negligible probability.

- Based on the signature property proven in Appendix A, if the check in step ⑤ passes, then the light client can be sure that sufficiently many validators in $\mathcal{V}^{(e_1)}$ have individually signed the chain of messages $e_1, e_1 + 1, \dots, j_1 - 1, (j_1, \text{NextK}^{(j_1)})$ —except with negligible probability.
- The same holds for the subsequent subperiods.
- Due to majority-honesty, if enough validators signed, then the aggregated public keys at the break points are all the correct ones.
- If, in particular, the final provided set of keys—that is, $\text{NextK}^{(j_r)}$ or $\text{NextK}^{(j_r-1)}$ —is correct, then it can safely be used to verify the header.
- Finally, the state read off of a correct block header must be a correct state.

Thus, for our check to pass even though the final state was a false one, one of our assumptions (round-wise majority-honesty of validators or signature security) would have to be broken. This completes the proof. $\square$